

HOLLOW KNIGHT: PABLO

Luca Bertoldini, Andrea Frigerio,
Lorenzo Giannattasio, Chiara Gianola
4°AI 13/05/2026

INTRODUZIONE

Hollow Knight Pablo è un videogioco d’azione e avventura di tipo metroidvania.

Il gioco è ispirato a Hollow Knight, ambientato in un regno oscuro e misterioso popolato da creature ostili e antiche rovine.

Il protagonista, Pablo, dovrà esplorare ambientazioni pericolose, affrontare nemici sempre più forti e scoprire i segreti nascosti del regno, migliorando progressivamente le proprie abilità e il proprio equipaggiamento.



GAMEPLAY

Il gameplay combina esplorazione, combattimento e platforming.
Il giocatore controlla Pablo esplorando diverse aree della mappa per esplorare il regno di Hallownest, affrontando nemici e superando ostacoli ambientali.

Durante l'avventura è possibile muoversi liberamente, saltare tra le piattaforme e combattere utilizzando attacchi corpo a corpo.

Nel corso dell'avventura il giocatore ottiene nuove abilità che permettono di raggiungere aree prima inaccessibili e affrontare boss sempre più difficili.
Il gameplay include inoltre collisioni, gestione della fisica, sistema di vita e animazioni dinamiche per rendere l'esperienza più fluida e coinvolgente.

TECNOLOGIE UTILIZZATE

- Linguaggio di programmazione: Java (libGDX)
- Ambiente di sviluppo: IntelliJ
- Software per level design: Tiled
- Software per modifica assets: Figma
- Controllo di versione: GitHub



NEMICI

Crawlid

Piccolo nemico terrestre che si muove continuamente avanti e indietro nelle piattaforme.
Quando raggiunge un muro oppure il bordo della piattaforma cambia direzione automaticamente.



Tiktik

Piccolo nemico veloce che si muove continuamente sulle superfici. È uno dei nemici più semplici del gioco ma può risultare fastidioso in gruppo.



NEMICI

Husk Bully

Nemico terrestre più resistente rispetto agli altri nemici base. Si muove verso il giocatore e provoca danni tramite contatto.



Husk Hornhead

Nemico veloce che attacca il giocatore frontalmente. Utilizza movimenti rapidi e può cambiare direzione quando incontra ostacoli.



Husk Warrior

Nemico armato che utilizza attacchi corpo a corpo contro il giocatore. È più pericoloso dei nemici base grazie alla maggiore vita e ai danni elevati.



NEMICI

Vangefly

Nemico volante che segue il giocatore dall’alto.
Può cambiare posizione rapidamente e attaccare in aria.



FalseKnight

Boss pesante e aggressivo che attacca il giocatore utilizzando colpi molto potenti e movimenti lenti ma dannosi.



STRUTTURA PROGETTO

Ogni nemico ha un package dedicato con 2 classi:

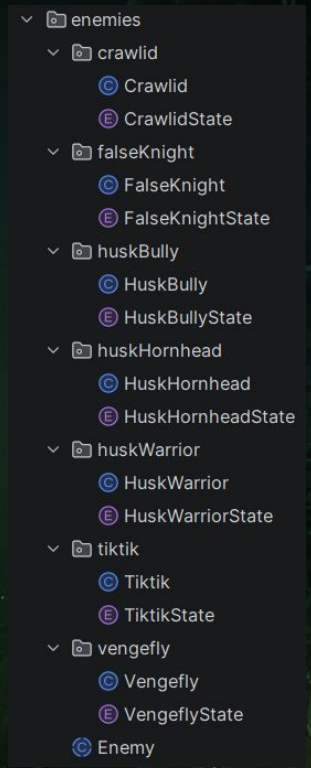
- La CLASSE PRINCIPALE gestisce il funzionamento del nemico
- La CLASSE DEGLI STATI gestisce i vari comportamenti del personaggio

CLASSE PRINCIPALE

- Movimento
- Gravità
- Collisioni
- Inseguimento del giocatore
- Attacchi
- Sistema di danno
- Sistema di morte
- Knockback
- Rotazione
- Sensori e rilevamento del terreno
- Cambio direzione
- Gestione della vita
- Gestione della fisica
- Movimento volante e terrestre
- Aggiornamento delle animazioni
- Camminata
- Idle
- Rotazione
- Attacco
- Danno subito
- Morte in aria
- Impatto al suolo
- Corpo finale a terra
- Movimento volante
- Cambio direzione
- Knockback
- Caduta
- Volo
- Animazioni dei boss

CLASSE STATI

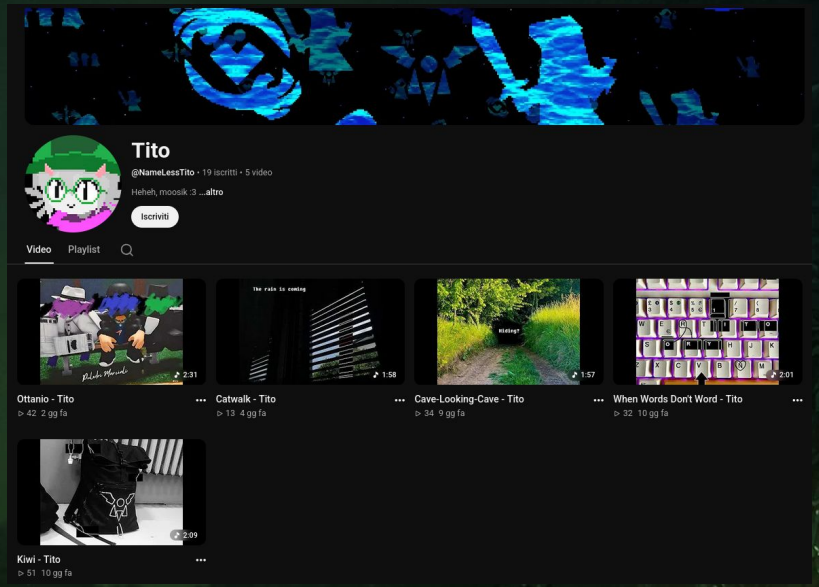
- Idle
- Patrol
- Movimento
- Turning
- Inseguimento
- Attacco
- Hit Stop
- Knockback
- Danno subito
- Dead Air
- Dead Land
- Morte
- Volo
- Collisione



OST

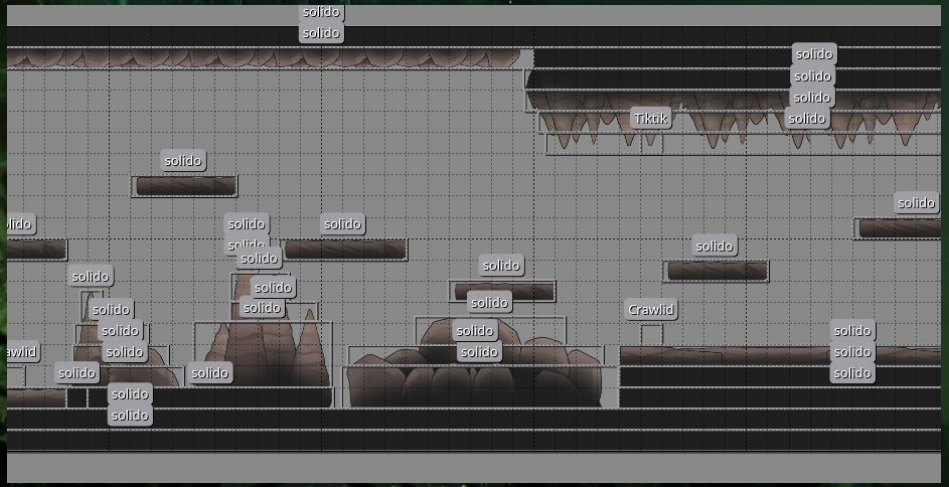
Tutte le musiche (OST) sono state interamente realizzate da una nostro amico in 4° AA che ha utilizzato il software “FL Studio” e tutta la sua esperienza su Online Sequencer.

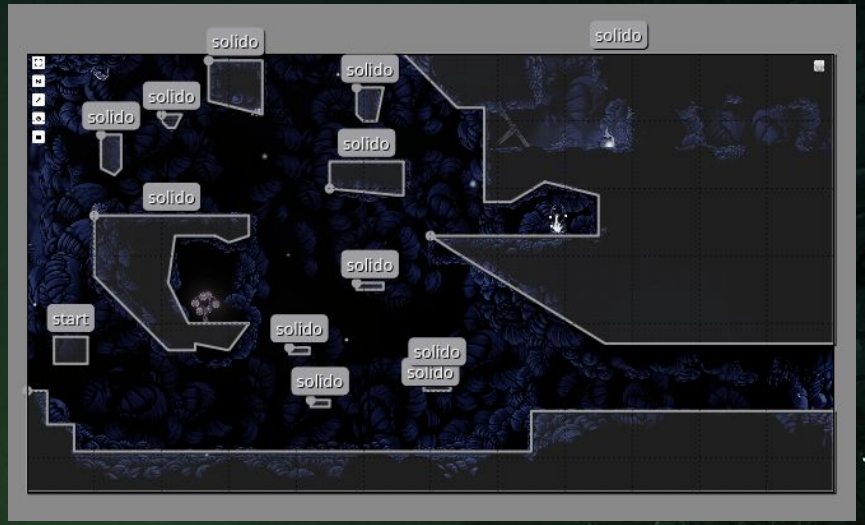




MAPPE







MAP GRAPH

MapGraph gestisce il collegamento tra i livelli tramite un registro di nodi che associa ogni mappa ai suoi vicini laterali.

Utilizzando riferimenti testuali per evitare errori di memoria, il sistema crea una struttura lineare e bidirezionale che permette al gioco di sapere sempre quale mappa caricare quando il giocatore raggiunge un bordo. In questo modo, singoli file separati vengono trasformati in un unico mondo di gioco continuo e navigabile.

MapGraph.java

```java
public class MapGraph
{
    public static class MapNode{...}

    private static final Map<String, MapNode> NODES = new HashMap<>();

    static
    {
        // Definizione Nodi
        MapNode map1 = new MapNode("assets/Maps/Mappa1.tmx");
        MapNode map2 = new MapNode("assets/Maps/Mappa2.tmx");
        MapNode map3 = new MapNode("assets/Maps/Mappa3.tmx");
        MapNode map4 = new MapNode("assets/Maps/Mappa4.tmx");

        // Connessioni
        map1.rightNeighbor = map2.path;
        map2.leftNeighbor  = map1.path;
        map2.rightNeighbor = map3.path;
        map3.leftNeighbor  = map2.path;
        map3.rightNeighbor = map4.path;
        map4.leftNeighbor  = map3.path;

        // Registrazione        NODES.put(map1.path, map1);
        NODES.put(map2.path, map2);
        NODES.put(map3.path, map3);
        NODES.put(map4.path, map4);
    }

    /** Restituisce il nodo associato al path, o null se non registrato. */
    public static MapNode get(String mapPath)    {...}

    /** True se esiste un vicino a destra per questa mappa. */
    public static boolean hasRight(String mapPath){...}

    /** True se esiste un vicino a sinistra per questa mappa. */
    public static boolean hasLeft(String mapPath){...}
}
```

TITLE SCREEN

TitleScreen.java

```java
public class TitleScreen extends BaseScreen
{
    // Variabili del metodo

    @Override
    public void initialize(){...}

    @Override
    public void update(float dt){...}

    @Override
    public void render(float dt)
    {
        // Parte iniziale metodo

        // Carica il nuovo frame solo quando cambia (evita di ricaricare ogni render)
        if (frameIndex != currentFrameIndex)
        {
            // I frame reali sono in assets/Titolo/ con nome frame_0001.png, ecc.
            String path = String.format("assets/Titolo/frame_%04d.png", frameIndex);

            if (Gdx.files.internal(path).exists())
            {
                Texture next = new Texture(Gdx.files.internal(path));

                if (currentFrame != null)
                    currentFrame.dispose();

                currentFrame      = next;
                currentFrameIndex = frameIndex;
            }
        }

        Gdx.gl.glClearColor(0, 0, 0, 1);
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT);

        if (currentFrame != null)
        {
            mainStage.getViewport().apply();
            float vw = mainStage.getViewport().getWorldWidth();
            float vh = mainStage.getViewport().getWorldHeight();
            batch.setProjectionMatrix(mainStage.getCamera().combined);
            batch.begin();
            batch.draw(currentFrame, 0, 0, vw, vh);
            batch.end();
        }
    }

    private void goToMenu(){...}

    @Override
    public void dispose(){...}
}
```

TitleScreen gestisce l'animazione d'apertura caricando ciclicamente 240 frame.

Per preservare la memoria video, il sistema carica un solo fotogramma alla volta e distrugge immediatamente quello precedente.

La schermata supporta lo skip universale tramite qualsiasi tasto o tocco e adatta automaticamente le immagini alla risoluzione dello schermo, garantendo una transizione fluida verso il menu principale al termine del video o su comando dell'utente.

LEVEL SCREEN - update()

LevelScreen gestisce la transizione fluida tra i livelli monitorando costantemente la posizione del giocatore rispetto ai confini della mappa.

Quando il personaggio raggiunge un bordo laterale, il sistema consulta il MapGraph per identificare la mappa adiacente; se il passaggio è consentito, salva i dati correnti di salute e anima tramite PlayerState e carica il nuovo scenario.

Questo meccanismo assicura la continuità dell'esperienza, permettendo al giocatore di mantenere i propri progressi e di apparire automaticamente sul lato corretto della nuova area di gioco.

LevelScreen.java

```java
public void update(float dt)
{
    // Metodi precedenti

    // Transizione mappa destra
    MapGraph.MapNode node = MapGraph.get(mapPath);

    if (node != null && node.rightNeighbor != null)
        if (pablo.getX() + pablo.getWidth() >= BaseActor.getWorldBounds().width - 2f)
        {
            PlayerState.saveFrom(pablo);
            BaseGame.setActiveScreen(new LevelScreen(node.rightNeighbor, "left"));
        }

    // Transizione mappa sinistra
    if (node != null && node.leftNeighbor != null)
        if (pablo.getX() <= 2f)
        {
            PlayerState.saveFrom(pablo);
            BaseGame.setActiveScreen(new LevelScreen(node.leftNeighbor, "right"));
        }

    logEnemyRuntimeState(dt);
}
```

LEVEL SCREEN - loadTextureSafe()

Il metodo loadTextureSafe garantisce la stabilità del gioco gestendo in modo sicuro il caricamento delle risorse grafiche ed evitando crash in caso di file mancanti.

Se il percorso di una texture non risulta valido, il sistema cattura l'eccezione e genera istantaneamente tramite Pixmap un quadratino colorato di emergenza come segnale visivo.

Questa tecnica di fallback permette agli sviluppatori di testare il software anche con asset incompleti, mantenendo l'applicazione attiva e sostituendo gli errori fatali con placeholder grafici temporanei che vengono caricati direttamente sulla GPU.

LevelScreen.java

```
private Texture loadTextureSafe(String path, Color fallback)
    {
        try
        {
            return new Texture(Gdx.files.internal(path));
        }
        catch (Exception e)
        {
            Gdx.app.log("LevelScreen", "Asset non trovato: " + path);
            Pixmap pix = new Pixmap(1, 1, Pixmap.Format.RGBA8888);
            pix.setColor(fallback);
            pix.fill();
            Texture t = new Texture(pix);
            pix.dispose();
            return t;
        }
    }
```

LEVEL SCREEN - createEmptyVesselTexture()

LevelScreen.java

```
private Texture createEmptyVesselTexture(int size)
    {
        Pixmap pix = new Pixmap(size, size, Pixmap.Format.RGBA8888);
        pix.setColor(0f, 0f, 0f, 0f);
        pix.fill();

        int cx     = size / 2;
        int cy     = size / 2;
        int radius = Math.max(1, (size / 2) - 2);

        pix.setColor(0.12f, 0.12f, 0.12f, 1f);
        pix.fillCircle(cx, cy, radius - 2);

        pix.setColor(0.35f, 0.35f, 0.35f, 1f);
        pix.drawCircle(cx, cy, radius - 1);

        Texture t = new Texture(pix);
        pix.dispose();
        return t;
    }
```

Il metodo createEmptyVesselTexture genera l'interfaccia del ricettacolo dell'anima via codice senza usare file esterni.

Sfruttando la classe Pixmap, il sistema disegna in memoria una base circolare grigia e un bordo più chiaro su uno sfondo trasparente.

Questo approccio procedurale garantisce la massima scalabilità per ogni risoluzione e riduce il numero di asset nel progetto, ottimizzando le risorse attraverso il rilascio immediato della memoria dopo la creazione della texture.

PABLO - updateJumpAnimation()

Il metodo updateJumpAnimation gestisce i frame del personaggio durante il salto, collegandoli direttamente alla fisica del movimento.

Invece di usare un'animazione a tempo fisso, il codice calcola una percentuale di completamento del salto basata sul rapporto tra il tempo trascorso e la durata totale della parabola. Questa progressione viene divisa in quattro fasi: il decollo iniziale, la fase di ascesa, la discesa e l'atterraggio finale.

Questo approccio garantisce che l'aspetto visivo sia sempre sincronizzato con la velocità e la gravità impostate, offrendo al programmatore un controllo totale sulla fluidità del movimento senza dover modificare i file grafici originali.

Pablo.java

```java
private void updateJumpAnimation()
    {
        float p = jumpTimer / totalJumpTime;

        if (p < 0.10f)
        {
            setAnimation(jumpUp);
        }
        else if (p < 0.50f)
        {
            float r = (p - 0.10f) / 0.40f;

            if      (r < 0.40f) setAnimation(frameRise1);
            else if (r < 0.80f) setAnimation(frameRise2);
            else                setAnimation(frameAltezzaMax);
        }
        else if (p < 0.90f)
        {
            float d = (p - 0.50f) / 0.40f;

            if      (d < 0.20f) setAnimation(frameAltezzaMax);
            else if (d < 0.60f) setAnimation(frameDrop1);
            else                setAnimation(frameDrop2);
        }
        else
        {
            setAnimation(frameDrop2);
        }
    }
```

ENEMY CLASS - FalseKnight

FalseKnight.java

```java
public class FalseKnight extends BaseActor
{
    // Varie variabili e costanti

    public FalseKnight(float x, float y, Stage s, Pablo pablo) {...}

    @Override
    public void act(float dt){...}

    private void loadAnimations(){...}

    private void tickIdle(){...}

    private void tickPatrol(){...}

    private void tickTurn(){...}

    private void tickAnticipateAttack(){...}

    private void tickAttack(){...}

    private void tickRecoverAttack(){...}

    private void tickJumpAnticipate(){...}

    private void tickJumpUp(){...}

    private void tickJumpAttackFall(){...}

    private void tickJumpLand(){...}

    private void tickStagger(){...}

    private void tickDead(){...}

    // Altri metodi
}
```

FalseKnight gestisce il boss principale attraverso una macchina a dodici stati che coordina tredici animazioni distinte.

Il sistema alterna dinamicamente inseguimento e attacchi (terrestri o in salto) basandosi sulla distanza dal giocatore, utilizzando tecniche come la duplicazione dei frame per enfatizzare l'anticipazione dei colpi. Ogni transizione di stato resetta i timer interni per garantire la sincronia tra logica e grafica, integrando effetti di hitstop e scuotimento dello schermo per aumentare il feedback dei colpi.

La classe controlla infine ogni dettaglio fisico, dal ribaltamento dello sprite (flip) alla gestione della gravità aumentata durante gli attacchi, fino alla dissolvenza finale dopo la sconfitta.

CONCLUSIONE

Durante lo sviluppo del progetto abbiamo cercato di ricreare le principali meccaniche di Hollow Knight, concentrandoci soprattutto sul gameplay, sulle animazioni e sul comportamento dei nemici.

Abbiamo iniziato realizzando la struttura base del gioco e successivamente aggiunto le diverse funzionalità, come il sistema di combattimento, la gestione degli stati dei personaggi, le collisioni e le animazioni.

Nel corso dello sviluppo abbiamo incontrato alcune difficoltà, in particolare nella gestione dei movimenti dei nemici, ma siamo riusciti a risolvere migliorando progressivamente il progetto.

Questo lavoro ci ha permesso di approfondire l'utilizzo di LibGDX e di comprendere meglio come viene sviluppato un videogioco 2D strutturato.